

Testing Basics

Write Tests That Give You Confidence to Ship

■ LEARNING OUTCOME

By the end of this module you will write unit tests with Vitest, test React components with Testing Library, mock APIs, and set up E2E tests with Playwright.

■ Skills You Will Gain

- Write unit tests with Vitest
- Test React components with RTL
- Query elements by role and label
- Test interactions: click, type, submit
- Mock API calls in tests
- Set up Playwright for E2E testing

■ Tests are insurance for your code. A strong test suite lets you refactor confidently, catch regressions before users do, and ship faster. Frontend tests pay off within days.

SECTION 1

Vitest Setup and Unit Tests

```
npm install -D vitest jsdom @testing-library/react @testing-library/user-event
@testing-library/jest-dom // vite.config.js export default defineConfig({ plugins: [react()], test:
{ environment:'jsdom', setupFiles:['./src/test/setup.ts'], globals:true }, }); // setup.ts: import
'@testing-library/jest-dom'; // Unit tests for pure functions import { describe, it, expect, vi }
from 'vitest'; import { validateEmail, calculateTotal } from './utils'; describe('validateEmail',
() => { it('accepts valid emails', () => { expect(validateEmail('alice@example.com')).toBe(true);
expect(validateEmail('user+tag@domain.co.uk')).toBe(true); }); it('rejects invalid emails', () => {
expect(validateEmail('notanemail')).toBe(false); expect(validateEmail('')).toBe(false); }); });
describe('calculateTotal', () => { const items = [{ price:100, quantity:2 }, { price:50, quantity:3
}]; it('calculates subtotal', () => expect(calculateTotal(items)).toBe(350)); it('applies
discount', () => expect(calculateTotal(items,10)).toBe(315)); it('returns 0 for empty cart', ()=>
expect(calculateTotal([])).toBe(0)); it('throws for negative qty', ()=>
expect(()=>calculateTotal([{price:100,quantity:-1}])).toThrow(); }); # Run tests npm test # watch
mode npm test -- --run # single run npm test -- --coverage # coverage report
```

SECTION 2

React Testing Library and Playwright

```
// Test BEHAVIOUR not implementation -- query as users find elements import { render, screen,
waitFor } from '@testing-library/react'; import userEvent from '@testing-library/user-event'; //
Query priority: getByRole > getByLabelText > getByText > getByTestId describe('SearchBar', () => {
it('renders search input and button', () => { render(<SearchBar onSearch={vi.fn()} />);
expect(screen.getByRole('searchbox')).toBeInTheDocument(); expect(screen.getByRole('button',
{name:/search/i})).toBeInTheDocument(); }); it('calls onSearch with typed value', async () => {
const user = userEvent.setup(); const mock = vi.fn(); render(<SearchBar onSearch={mock} />); await
user.type(screen.getByRole('searchbox'), 'laptop'); await user.click(screen.getByRole('button',
{name:/search/i})); expect(mock).toHaveBeenCalledWith('laptop'); }); it('displays products after
loading', async () => { vi.spyOn(window, 'fetch').mockResolvedValueOnce( new
Response(JSON.stringify([{id:1, name:'Laptop Pro', price:75000}])) ); render(<ProductList
category='electronics' />); await waitFor(() => expect(screen.getByText('Laptop
Pro')).toBeInTheDocument()); }); it('shows error when API fails', async () => {
vi.spyOn(window, 'fetch').mockRejectedValueOnce(new Error('Network error')); render(<ProductList
category='electronics' />); await screen.findByRole('alert');
expect(screen.getByRole('alert')).toHaveTextContent(/something went wrong/i); }); }); // PLAYWRIGHT
E2E npm install -D @playwright/test && npx playwright install // playwright.config.ts:
baseURL:'http://localhost:5173' test('user can log in', async ({ page }) => { await
page.goto('/login'); await page.getByLabel('Email').fill('alice@example.com'); await
page.getByLabel('Password').fill('password123'); await page.getByRole('button', { name:'Sign In'
}).click(); await expect(page).toHaveURL('/dashboard'); await expect(page.getByRole('heading',
{name:'Welcome'})).toBeVisible(); });
```

■ PRACTICE Q & A

Q1. What is the difference between unit, integration, and E2E tests?

A: Unit: test single function/component in isolation -- fast, many. Integration: multiple components together. E2E: whole app in real browser -- slow, few. Testing pyramid: many units, some integration, few E2E.

Q2. Why does RTL discourage testing implementation details?

A: Tests of internal state or method names break when you refactor, even if behaviour is unchanged. RTL tests what USER sees and does -- tests only break when behaviour changes.

Q3. What is the difference between getBy, queryBy, and findBy?

A: getBy: throws if not found (sync). queryBy: returns null (sync, for asserting absence). findBy: returns Promise, waits for element (async rendering).

Q4. What is vi.fn()?

A: Creates a mock function recording calls. Assert: fn was called, with what args, how many times. vi.spyOn() replaces existing method with a mock.

Q5. What should you test in React components?

A: Rendering (correct content), interactions (click/type does right thing), async states (loading/error/success), accessibility. Don't test: styling, implementation details.

■ Testing Cheat Sheet	
<code>describe('name', () => {})</code>	Group related tests
<code>it('should...', async () => {})</code>	Single test case
<code>expect(val).toBe(expected)</code>	Equality assertion
<code>expect(fn).toHaveBeenCalled(arg)</code>	Mock call assertion
<code>render(<Component />)</code>	Render component
<code>screen.getByRole('button', {name:/txt/i})</code>	Query by ARIA role
<code>screen.getByLabelText('Email')</code>	Query by label
<code>screen.findByText('Loaded')</code>	Async query
<code>userEvent.type(el, 'text')</code>	Simulate typing
<code>userEvent.click(button)</code>	Simulate click
<code>waitFor(()=>expect(el).toBeVisible())</code>	Wait for async
<code>vi.spyOn(window, 'fetch').mockResolvedValueOnce</code>	Mock API call